

«НЕ», ПРОСТО СЛОВО

Отрицание в графе соседств

varshchik · me@varshchik.dev · github.com/varshchik

РЕЗЮМЕ

В большинстве систем символического рассуждения отрицание реализовано как отдельный примитив: унарный оператор $\neg$, флаг полярности, специальное отношение. NSAI — граф соседств слов, где у каждого ребра хранятся позиции его вхождений в источниках, — не делает с «не» ничего особенного: слово «не» живёт в графе наравне с любым другим словом. В заметке разбирается, что из этого следует на примере с четырьмя предложениями про спящего кота, и формулируется методологический принцип: прежде чем кодировать наблюдаемое поведение как примитив, стоит проверить, не появляется ли оно побочно из более простой механики, уже имеющейся в системе — или не относится ли оно вовсе к задаче, которую система решает.

Сразу оговорю границу, чтобы не обещать лишнего. Речь идёт о дискавери — о том, чтобы показать, какие связи между словами есть в корпусе и где они сказаны прямо. Это не вывод истины и не логическое рассуждение; почему — в разделах 5 и 6. И, как сказано ниже, объект заметки — не «отрицание вообще», а конкретное слово «не».

1. ВВЕДЕНИЕ

Несколько месяцев назад я возился с обработкой отрицания в NSAI — небольшой исследовательской системе для работы с текстами — и поймал себя на странной мысли. Естественный план был такой: завести оператор НЕ, добавить флаг полярности к фактам, написать ветку обработки для запросов с отрицанием. Так делают почти все системы такого типа: классическая логика построена вокруг унарного $\neg$ [1], экспертные системы 80–90-х писали явный NOT [3], современные knowledge graphs ставят специальные отношения для отрицательных утверждений [4]. Я начал писать эту ветку и где-то посредине заметил, что не понимаю, зачем именно я её пишу.

Вопрос звучал примерно так. Откуда берётся убеждённость, что отрицание нуждается в собственном механизме? Из языка — в нём есть «не». Из логики — в ней есть $\neg$. Но это связка вида «X есть в наблюдаемом феномене $\rightarrow$ X должен быть и в реализации». А такая связка вовсе не самоочевидна. Из того, что вода течёт, не следует, что в водопроводе должен быть модуль flow(). Из того, что в физике массы притягиваются, не следует, что в симуляторе должна быть подпрограмма, реализующая притяжение как примитив. Поведение, наблюдаемое в системе, и механика, его порождающая, — разные вещи, и спутывать их — типовая категориальная ошибка, особенно лёгкая, когда строишь то, что должно демонстрировать определённое поведение.

Я выключил то, что писал, и попробовал думать с другой стороны. Что, если отрицание — не примитив, а наблюдаемое поведение? Тогда вопрос становится не «как реализовать оператор НЕ», а «какие более простые механизмы могут дать поведение отрицания как побочный эффект». Это другой класс задачи. И когда я начал её решать всерьёз, ответ оказался смущающе коротким: ничего специально реализовывать не нужно. То, как NSAI устроен на самом базовом уровне — граф соседств слов с привязкой рёбер к позициям в тексте, — уже даёт нужное поведение, если не мешать ему специальными костылями. Эта заметка — про то, как и почему.

2. АРХИТЕКТУРА

Прежде чем описывать механику, стоит договориться об объекте. Что такое отрицание — вопрос без единого ответа, и это не недосмотр, а свойство самого понятия. В логике отрицание определяется через истинность: $\neg A$ истинно, когда A ложно. Но это определение применимо лишь там, где есть истинностные значения, а система на чистой форме ими не располагает (раздел 6). В лингвистике

«отрицание» — скорее семейство разнородных средств (частица, аффикс, антоним, имплицитный отказ), объединённых не общей сущностью, а общим ярлыком. Поэтому корректнее сразу сузить: ниже речь не об «отрицании вообще», а о конкретном слове «не» — о том, как токен «не» ведёт себя в графе соседств наравне с любым другим токеном. Всё «поведение отрицания», которое здесь обсуждается, — это поведение этого узла и его связей, а не операция над истинностью.

Архитектура чрезвычайно простая, и в её простоте — половина рассуждения. Текст поступает по одному фрагменту за раз. Фрагмент разбирается на токены, и для каждой пары непосредственных соседей записывается ребро в граф. Параллельно для каждого ребра запоминается, **в каких позициях каких источников** оно встретилось: пара (источник, индекс). Никакой лемматизации, никаких POS-тегов, никаких склеек частиц с предикатами — слова идут в граф как они есть. Узел — слово. Ребро — «это слово стояло рядом с этим». У ребра — список позиций, где это наблюдалось.

Замечание о версии. В более ранней версии (v0.1 / v2) близость рёбер определялась через общий `ctx_id` — идентификатор предложения, к которому привязывалось ребро. В текущей версии (v2.1) примитив `ctx_id` убран: вместо принадлежности к предложению хранятся позиции (источник, индекс), а близость вычисляется как «позиции укладываются в окно заданной ширины в одном источнике». Это не косметика: ширина окна стала **параметром запроса**, а не зашитой границей предложения. Узкое окно (1–3) — тесные локальные связи; широкое (десятки) — связи через расстояние. Содержательно поведение, описанное ниже, от этой замены не изменилось; изменилось то, что оно больше не привязано к пунктуации и к понятию «предложение».

На таком хранилище живёт операция, которая для нас и важна: классификация связи между двумя словами. Найдя путь между двумя узлами, система смотрит на позиции рёбер этого пути. Если есть один источник, в котором все рёбра пути укладываются в окно заданной ширины, — связь помечается как **найденно напрямую**: в корпусе есть место, где она сказана подряд, и его можно процитировать. Если такого источника нет — путь собран из разных мест, и переход проходит через узел, где источник/окно меняется. Такой путь помечается меткой **выведено из связей**.

Имена выбраны намеренно так, чтобы описывать **операцию**, а не приписывать ей статус. «Найдено напрямую» — не «истинно» и не «утверждается как факт»; это значит ровно «такая последовательность лежит в одном источнике подряд». «Выведено из связей» — не логический вывод, не дедукция и не силлогизм; это значит «цепочка собрана переходом по рёбрам из разных мест». Система ничего не выводит в смысле *logical reasoning* и ничего не утверждает об истинности — она лишь различает «сказано где-то подряд» и «склеено из разных мест». Прежние имена («утверждение» / «рассуждение») этого различия не отражали: они обещали эпистемический и логический статус, которого в механике нет. Назвать структурную операцию словом, несущим такое обещание, — отдельная ошибка, родственная той категориальной путанице, с которой начато введение (поведение против механики); здесь это путаница между **именем** и механикой. Поэтому ниже используются имена, равные тому, что система делает.

3. ДЕМОНСТРАЦИЯ

Чтобы проверить, что отрицание не требует отдельного обращения, я загрузил четыре простых факта про кота, каждый — отдельным источником: «Кот спит на диване», «Кот спит днём», «Кот не спит ночью», «Кот не спит когда голоден». В графе появились узлы кот, спит, на, диване, днём, не, ночью, когда, голоден — каждое слово самостоятельным узлом, без склейки `не_спит` и без лемматизации. Различие между «спит» и «не спит» живёт **не в узлах**, а в том, из каких источников приходят рёбра вокруг узла спит.

Вот реальный вывод текущей версии на запрос «кот — спит»:

```
кот — спит:
[найденно напрямую] кот → спит      «КОТ СПИТ»
[найденно напрямую] кот → не → спит  «КОТ НЕ СПИТ»
```

Система не выбирает между «кот спит» и «кот не спит». Она находит **оба** пути, помечает **оба** как найденные напрямую и для каждого показывает, где это сказано. С точки зрения дискавери оба верны: корпус действительно содержит и то, и другое; противоречия нет, потому что условия разные (днём / на диване против ночью / когда голоден). Будь у системы оператор отрицания, ей пришлось бы выбирать. Здесь выбора нет, потому что нет

двоичной полярности — есть два разных пути от кот к спит, и оба валидно показываются.

Запрос «кот — ночью»:

```
кот — ночью:  
[выведено из связей] кот → спит → ночью (нет общего окна)  
[найдено напрямую] кот → не → спит → ночью «кот не спит ночью»
```

Тут работает различие между «найдено напрямую» и «выведено из связей». Путь кот → спит → ночью существует структурно (оба ребра есть в графе), но рёбра приходят из разных источников: кот↔спит — оттуда, где кот спит, а спит↔ночью — оттуда, где он не спит. Общего окна нет, и путь помечается как склейка из разных мест. То есть «кот спит ночью» — **не то, что сказано в корпусе**, а домысленная сборка, и система это видит. А путь кот → не → спит → ночью целиком лежит в одном источнике, где так и сказано, «Кот не спит ночью», — и это найдено напрямую.

Различение работает **без всякого знания о том, что «не» — это отрицание**. Оно работает потому, что рёбра одного пути физически делят общее место в тексте, а у альтернативного — не делят.

Что зацепило меня здесь больше всего — полное отсутствие в коде чего-либо, что можно было бы назвать «обработкой отрицания». Нет узла не_спит. Нет правила «если перед глаголом частица — склейкай». Нет ветки if is_negative_query. Нет поля polarity. Узел не ничем не отличается от узла на или когда — просто слово, стоявшее рядом с другими. Различие между «кот спит» и «кот не спит» появляется только в момент запроса; до запроса в графе никакого различия нет, есть рёбра с привязкой к позициям. То, что классическая система обрабатывала бы как отрицание, здесь **не обрабатывается вообще** — и в дискавери это ничему не мешает, потому что дискавери не различает истину и ложь. «Не» не решено каким-то изящным способом; оно просто не участвует как отдельная операция, оставаясь обычным узлом.

4. ЧЕТЫРЕ ФОРМЫ ОТРИЦАНИЯ

Лингвистическая типология отрицания [5] выделяет несколько форм, и стоит проговорить их явно. Отрицание — это не только частица «не» перед глаголом. Есть антонимы: «горячий» и «холодный». Есть имплицитное отрицание: «промолчал», «отказался», «проигнорировал» — положительные

слова, в которых смысл отрицания зашит лексически. Есть модальные ослабления — «вряд ли», «едва ли» — частичные отрицания, которые в классическую бинарную логику не помещаются. И есть префиксные: «невнимательный», «несогласие» — отрицание внутри слова.

Система, где отрицание реализовано как оператор НЕ, оказывается заложницей одной из этих форм — той, под которую её настроили. Работает на частице — антонимы выпадают: для них нет грамматического маркера. Имплицитное выпадает по той же причине. Модальные ослабления выпадают, потому что булев оператор не умеет градуацию. А с префиксами начинаются вопросы вида «считать ли „невнимательный“ отрицанием „внимательный“», и любой ответ создаёт несоответствие с обработкой частицы. Чтобы покрыть все формы, приходится изобретать несколько механизмов и ещё один, чтобы их согласовать. Эта фрагментация подходов задокументирована в обзорах [6].

В графе соседств все эти формы попадают на один и тот же уровень обработки — **уровень обхода** — потому что ни одна из них не обрабатывается отдельно. Маршрут, которым каждое слово попало в граф, разный, но на уровне обхода он неразличим. Узел есть узел; обходится одинаково; его окрестность накапливается одинаково; различие «сказано прямо» и «склеено из разных мест» делается одинаково. Узлы спит и не соседствуют — потому что слова стояли рядом. Узел промолчал накопит свою окрестность, через которую система покажет цепочки к «не сказал», «не ответил», — но это будут пути с меткой «выведено из связей» (склейка из разных мест), не найденные напрямую.

Здесь важно **не приписать системе того, чего она не делает**. Про антонимы у меня в прежней версии этой заметки было сказано, что узлы «холодный» и «тёплый» «будут противопоставляться через разные пересечения контекстных множеств». Это неверно, и стоит исправить прямо. Граф соседств **не отличает антонимы от синонимов**: слова, конкурирующие за одну позицию в фразе («спит» / «не спит», «горячий» / «холодный»), имеют похожие окрестности — ровно потому, что употребляются в похожем окружении. Это давно известное свойство дистрибутивной семантики: соседства улавливают *ассоциативную близость* (relatedness), а не *смысловое тождество*

(similarity) [7, 8]; для разведения этих вещей создавались отдельные ресурсы и методы [16, 17].

Но для **дискавери это не нужно**. Система не утверждает, что «спит» и «не спит» — синонимы или антонимы; она показывает, что у не есть своя окрестность (кот, спит) и что путь не → спит найден напрямую, опираясь на конкретные источники. Назвать это противопоставлением или нет — решает тот, кто читает вывод, а не граф. Различение «антоним против синонима» — отдельная задача с собственными допущениями; NSAI её не решает и не обязан. Стоит сказать точнее, почему *не обязан*: противоположность — не свойство слова, а отношение, которое накладывает наблюдатель. Чтобы подумать о «земле», ум не вызывает сперва «небо»; узел значит через свою окрестность употреблений, а не через антипод. Оппозиция, если она кому-то нужна, вносится запросом, а не хранится в графе. Поэтому одна механика обхода работает со всеми этими словами одинаково — на уровне «что с чем связано и где это сказано», — потому что ни одну из форм отрицания она не обрабатывает как отрицание. На этом уровне она и останавливается.

5. МЕТОДОЛОГИЧЕСКИЙ УРОК ШИРЕ ОТРИЦАНИЯ

Урок формулируется так: прежде чем кодировать наблюдаемое поведение как внутренний примитив, стоит спросить, не появляется ли оно как побочный эффект чего-то более простого, уже имеющегося в системе. Удивительно часто ответ — да. И тогда правильный шаг — не добавить механизм, а отказаться от идеи, что он нужен.

В архитектуре NSAI тот же фильтр сработал в нескольких местах, и каждый раз вёл к удалению, а не к добавлению.

- **Идентичность** — нет отдельного механизма «это та же сущность»; узел есть узел, операция идентификации сверху не нужна.
- **Различение прямого и косвенного** — нет компонента-валидатора; разница между «сказано подряд в одном источнике» и «склеено из разных» вытекает из того, делят ли рёбра пути общее окно.

Стоит назвать и **обратную сторону** этого принципа — места, где «удаления» недостаточно и нужное поведение **не возникает** само. Это важно, чтобы

фильтр не превратился в догму «механизм никогда не нужен».

- **Конфликт между фактами**. Система не находит противоречие сама. «Кот спит» и «кот не спит» лежат в графе рядом как два найденных напрямую факта; то, что это *конфликт*, — наблюдение читателя, наложенное на вывод, а не операция внутри системы. Для дискавери этого достаточно (показать оба и источники), но называть это «обработкой конфликта» было бы преувеличением: обработки нет, есть отсутствие выбора.
- **Вывод и пересмотр**. Граф аддитивен: каждое новое наблюдение только добавляет вес связи, ничего не отменяя. Поэтому поведение, требующее *немонотонности* — когда новый факт отменяет прежний вывод (belief revision), — из такой структуры **не возникает** и не должно. Это не дыра, которую надо залатать внутри NSAI; это граница задачи (см. раздел 6).
- **Причинность**. Сובлазн здесь сильнее, чем кажется, потому что у рёбер графа есть позиции, а значит — порядок: «вспышка» раньше «грома» в тексте. Отсюда один шаг до того, чтобы сделать ребро направленным и назвать направление причинностью. Шаг этот ложный, и стоит проговорить почему. Во-первых, порядок упоминания — не порядок событий: тот же факт можно изложить с конца («грянул гром, а перед ним — вспышка»). Во-вторых, даже верный порядок событий не есть причинность: вспышка и гром — два следствия одного разряда, а не причина и следствие, и читаемая «цепь» здесь — иллюзия, навязанная порядком. Глубже: причинность вообще не содержится в наблюдении. Различие между «нажал кнопку — закрылась вкладка» (причина, которую знаешь, потому что *вмешивался* и видел исход) и «вспышка — гром» (только смотрел) лежит не в словах, а в опыте вмешательства — в том, что Pearl [19] отделяет как $\text{do}(\cdot)$ от обычного наблюдения. Текст — протокол наблюдений; вмешательств в нём нет. Поэтому «А рядом с В в таком порядке» не отличает причину от со-следствия, от совпадения, от обратной связи — все они в графе выглядят одинаково, как ребро между узлами. Со-occurrence не несёт этого различия, и никакое усложнение обхода его не добудет: оно не в данных. Честный ход тот же, что и выше,

— не выводить причинность из смежности, а оставить связь голой («вот что рядом и где сказано»), отдав интерпретацию читателю, у которого есть модель мира. Это, кстати, ровно то место, где беглость языковых моделей вводит в заблуждение: на «Х. Почему?» модель уверенно продолжит «потому что Y», но это не обнаруженная причина, а частотное продолжение — постфактумная подгонка под уже данную пару, а не вывод вперёд. Граф в этом месте молчит; молчание здесь честнее гладкого ответа.

Каждый случай первого рода устроен по одному шаблону: классическая архитектура постулирует отдельный компонент, NSAI обходится без него, и поведение всё равно наблюдается — потому что нужная асимметрия уже есть в данных, в распределениях, в том, из каких источников приходят рёбра. Случаи второго рода показывают границу: когда асимметрии в данных нет (направление, причинность, несовместимость), поведение не появляется, и честный ход — вынести его за пределы того, что система обещает.

Среди случаев первого рода отрицание — самый наглядный: соблазн «реализовать» особенно велик, потому что тысячи лет логической традиции говорят, что отрицание фундаментально. Стоит добавить, что и **внутри самой логики** отрицание-примитив не единственно возможно. Routley & Routley [18] разделяют *функцию* отрицания и его конкретную реализацию и показывают, что классические законы (например, «из противоречия следует всё») в релевантных и параконсистентных логиках не обязаны выполняться. Это не моя позиция и не поддержка отказа от отрицания — Routley & Routley говорят о *разных теориях* отрицания, а не об отказе от него. Но это независимое свидетельство, что «отрицание = унарный →» — конвенция, а не данность. NSAI делает родственный, но более радикальный для своей задачи шаг: убирает оператор целиком, оставляя поведение эмерджентным — там, где этого хватает.

6. ШИРЕ NSAI: ФОРМАЛИЗМ В ОБРАБОТКЕ ЯЗЫКА

Тот же фильтр работает на ступень выше — на уровне подхода к языку. Этот сдвиг видно наглядно, потому что у проекта есть две версии: v1 (она же v0.1) и v2 лежат рядом в репозитории. v1 использовала весь стандартный аппарат для русского текста: POS-тегирование через natasha [14], лемматизацию

через rymorphy3 [13], дерево синтаксических зависимостей, типизированные факты вида (предикат, [аргументы]). Каждый компонент казался необходимым.

При переходе к v2 каждая стадия прошла тот же тест, что и отрицание: даёт ли формальный компонент поведение, которого без него нет, — или поведение возникает и так, а компонент навязан традицией?

Ответ оказался везде один. **POS-тегирование** не нужно: если два слова стоят рядом, их совместность фиксируется независимо от частей речи. **Лемматизация** не нужна: разные формы слова — это разные узлы со своими окрестностями, и то, что они «варианты одного слова», либо проявится как сходство окрестностей, либо нет — и в обоих случаях это наблюдение из данных, не аксиома сверху. **Синтаксический парсер** не нужен для той классификации, которую делает система: достаточно знать, лежат ли рёбра пути в общем окне. **Типы фактов** не нужны: ребро есть ребро, дальнейшее различие делается той же проверкой общего окна.

Конкретно: v1 — около полутора тысяч строк с тяжёлыми зависимостями. v2 — около ста пятидесяти строк, чистый Python без внешних зависимостей. Но показательнее не число строк, а область применимости: v1 в принципе не могла работать с не-русским текстом, потому что весь её аппарат был привязан к natasha и rymorphy3. v2 работает с любым текстом, потому что узлы в ней — последовательности символов между пробелами, без предположений об их природе. Это не оптимизация — это смена того, что система **обещает** делать, и она стала возможной потому, что v1 обещала слишком много.

Стоит зафиксировать, что именно убрано. Лемматизация, POS, парсинг в v1 были **препроцессорами**: текст проходил через них и попадал в обработку уже преобразованным. Это была переводная прослойка между текстом и обработкой, обещавшая упростить вход и вместо этого зашивавшая в него допущения о конкретном языке. Убрать её — значит позволить обработке работать с языком как с **данными**, а не с предобработанным набором допущений.

И здесь же — главная граница всего подхода, которую честнее назвать прямо. Bender & Koller [10], восходя к проблеме символьного заземления [11], показывают: система, работающая только с

формой, не выучивает значение в строгом смысле. NSAI работает именно с формой и **не претендует** на семантику в их смысле. Из этого следует конкретное ограничение: система **не выносит суждения об истинности**. Если в корпусе «плоская земля» встречается чаще «круглой», это говорит только о составе корпуса, а не о форме планеты. Частота совместной встречаемости — не голос за истину; это след того, как слова *употреблялись*, в духе Фёрта [7]: *meaning is use*, и не более того. Поэтому NSAI — инструмент дискавери и трассировки («вот что сказано и где»), а не оракул и не верификатор. Суждение об истинности вносит человек, у которого есть доступ к миру; или отдельный слой с внешней опорой на факты — но это уже не граф соседств.

Это не возражение Bender & Koller, а уточнение скоупа: v2 не пытается выйти за пределы формы, потому и не сталкивается с проблемой заземления. Методологически подход не нов. Brooks [12] в робототехнике начала 1990-х аргументировал отказ от внутренних моделей мира; «the world is its own best model» — формулировка, близкая тому, что v2 делает для языка. Технически v2 наследует дистрибутивной гипотезе Firth [7] и Harris [8]; word2vec и потомки [9] реализуют её в векторном пространстве, v2 — в графовом, с тем отличием, что граф сохраняет привязку к источникам, а вектор её теряет. Newell & Simon [2] сформулировали *physical symbol system hypothesis* — позицию, которой v2 имплицитно оппонирует, показывая, что один её

типичный компонент (явные операторы) можно убрать без потери поведения **в пределах дискавери**.

7. ЧТО СДЕЛАНО НА ПРАКТИКЕ

Эту заметку стоит закончить не обещаниями, а тем, что реально добавлено в систему и работает.

В систему добавлена операция многословного поиска (*find*): по строке-запросу она находит места в корпусе, где *input*-слова сходятся в пределах окна, с привязкой к источнику и цитатой. Запрос «сократ платон» на небольшом корпусе находит фрагмент «сократ был учителем платона» и показывает его источник. Это и есть честная форма того, на что граф соседств имеет право: не ответ «да, преподавал» (это было бы суждением, которого система выносить не может), а «вот где в корпусе эти слова стоят рядом — смотри сам». Морфологическая близость по символьным *n*-граммам связывает форму запроса с формой в тексте (платон → платона) без лемматизатора; ценой служит то, что при низком пороге иногда подхватывается однокоренное чужое слово — приемлемо для инструмента, который *направляет внимание*, а не выносит вердикт.

Код, тесты и подробности архитектуры — на github.com/varshchik/nsai. Если в заметке есть места, где аргумент кажется хрупким или не сходится, — буду рад письму на me@varshchik.dev. NSAI — сольный исследовательский проект, и внешний взгляд для меня особенно ценен.

ССЫЛКИ

- [1] McCarthy, J. (1959). Programs with Common Sense. In *Mechanisation of Thought Processes*, Vol. 1. London: HMSO.
- [2] Newell, A. & Simon, H. A. (1976). Computer Science as Empirical Inquiry: Symbols and Search. *Communications of the ACM* 19(3), 113–126.
- [3] Buchanan, B. G. & Shortliffe, E. H. (1984). *Rule-Based Expert Systems: The MYCIN Experiments of the Stanford Heuristic Programming Project*. Addison-Wesley.
- [4] Hogan, A., Blomqvist, E., Cochez, M., d'Amato, C., de Melo, G., Gutierrez, C., et al. (2021). Knowledge Graphs. *ACM Computing Surveys* 54(4), Article 71.
- [5] Horn, L. R. (1989). *A Natural History of Negation*. University of Chicago Press. (Reprinted 2001, CSLI Publications.)
- [6] Morante, R. & Sporleder, C. (2012). Modality and Negation: An Introduction to the Special Issue. *Computational Linguistics* 38(2), 223–260.
- [7] Firth, J. R. (1957). A synopsis of linguistic theory, 1930–55. In *Studies in Linguistic Analysis*. Oxford: Blackwell, 1–32.
- [8] Harris, Z. S. (1954). Distributional Structure. *Word* 10(2–3), 146–162.
- [9] Mikolov, T., Sutskever, I., Chen, K., Corrado, G. & Dean, J. (2013). Distributed Representations of Words and Phrases and their Compositionality. *Advances in Neural Information Processing Systems* 26 (NIPS 2013).
- [10] Bender, E. M. & Koller, A. (2020). Climbing towards NLU: On Meaning, Form, and Understanding in the Age of Data. *Proceedings of the 58th Annual Meeting of the ACL*, 5185–5198.
- [11] Harnad, S. (1990). The Symbol Grounding Problem. *Physica D* 42(1–3), 335–346.
- [12] Brooks, R. A. (1991). Intelligence without representation. *Artificial Intelligence* 47(1–3), 139–159.
- [13] Korobov, M. (2015). Morphological Analyzer and Generator for Russian and Ukrainian Languages. In *Analysis of Images, Social Networks and Texts (AIST 2015)*, CCIS vol. 542. Springer, 320–332.
- [14] natasha — Solutions for basic NLP tasks for Russian. Software library. <https://github.com/natasha/natasha>
- [15] de Marneffe, M.-C., Manning, C. D., Nivre, J. & Zeman, D. (2021). Universal Dependencies. *Computational Linguistics* 47(2), 255–308.
- [16] Hill, F., Reichart, R. & Korhonen, A. (2015). SimLex-999: Evaluating Semantic Models with (Genuine) Similarity Estimation. *Computational Linguistics* 41(4), 665–695.
- [17] Lin, D., Zhao, S., Qin, L. & Zhou, M. (2003). Identifying Synonyms among Distributionally Similar Words. *Proceedings of IJCAI 2003*, 1492–1493.
- [18] Routley, R. & Routley, V. (1985). Negation and Contradiction. *Revista Colombiana de Matemáticas* 19(1–2), 201–231.
- [19] Pearl, J. (2009). *Causality: Models, Reasoning, and Inference* (2nd ed.). Cambridge University Press.